

Key-Value Store v7

Rebalancing e migrazione

Architetture dei Sistemi Distribuiti

Lezione 16

Nella lezione sullo sharding avevamo una situazione semplice:

- insieme di shard fisso;
- funzione di routing stabile;
- posizione teorica e posizione reale della chiave coincidenti.

Questa lezione introduce la prima vera forma di topologia dinamica:

- si aggiunge uno shard;
- il routing cambia subito;
- il dato non si muove da solo.

L'assunzione che si rompe

Con due shard:

$$\text{target} = \text{hash}(\text{key}) \bmod 2$$

Con tre shard:

$$\text{target} = \text{hash}(\text{key}) \bmod 3$$

Quindi ADD_SHARD non cambia solo la configurazione del cluster. Cambia il significato operativo di domande come:

- “dove devo scrivere questa chiave?”
- “dove mi aspetto di leggerla?”

Due nozioni da tenere separate

Da qui in poi una chiave ha almeno due “indirizzi”:

- **destinazione teorica**: lo shard scelto dalla funzione di routing corrente;
- **posizione reale**: lo shard che contiene davvero il valore in questo istante.

Con topologia statica le due nozioni coincidono quasi sempre. Con topologia dinamica possono divergere per un tempo arbitrario. Il punto non è solo implementativo: è un problema di contratto osservabile.

Scenario con due shard:

```
SET alpha one      -> OK shard=S0
SET gamma three    -> OK shard=S1
WHERE alpha        -> OK shard=S0 port=6441
WHERE gamma        -> OK shard=S1 port=6442
```

In questa fase:

- il router calcola un target;
- lo shard raggiunto e' quello che contiene il dato;
- il client non vede tensione tra topologia e contenuto.

Trace Dopo ADD_SHARD

Dopo l'aggiunta del terzo shard:

```
ADD_SHARD S2 127.0.0.1 6443 -> OK shard_added=S2  
WHERE alpha -> OK shard=S2 port=6443  
GET alpha -> NOT_FOUND
```

Questo e' un risultato didatticamente importante.

- il router sta usando la topologia nuova;
- il valore e' ancora nello shard vecchio;
- il sistema e' vivo, ma non sta mantenendo l'illusione di continuita'.

Il laboratorio rende esplicita la topologia dinamica con tre operazioni:

- ADD_SHARD: modifica l'insieme degli shard attivi;
- PLAN <key>: rende osservabile il target teorico corrente;
- REBALANCE: riallinea il dato alla topologia.

Sono interessanti perché separano intenzionalmente:

- cambio di configurazione;
- osservabilità del nuovo routing;
- spostamento effettivo dei dati.

```
def _select_shard(self, key, shards=None):  
    active = shards if shards is not None else self._shards  
    return active[stable_hash(key) % len(active)]  
  
def _handle_add_shard(self, argument_blob):  
    ...  
    with self._lock:  
        self._shards.append(ShardEndpoint(...))
```

- la funzione di routing dipende solo dalla lista corrente degli shard;
- ADD_SHARD aggiorna subito quella lista;
- nessuna logica di migrazione e' implicita in questa operazione.

Nel lab il rebalance e' intenzionalmente semplice:

- 1 LIST_ITEMS su ogni shard;
- 2 calcolo del target con la topologia corrente;
- 3 IMPORT_KEY sul nuovo shard;
- 4 DELETE_LOCAL sul vecchio shard.

IMPORT_KEY(key, value)

DELETE_LOCAL(key)

L'ordine e' rilevante: prima si copia, poi si cancella.

Perche' La Finestra Critica Esiste

Tra `ADD_SHARD` e fine di `REBALANCE` possono esistere stati come:

- routing nuovo, dato ancora vecchio;
- dato copiato su due shard;
- cancellazione parziale in caso di errore a meta' protocollo.

La safety che ci interessa qui non e' "nessun thread entra in una sezione critica". E' piuttosto:

- non perdere chiavi;
- non mentire sul significato di una lettura;
- non confondere destinazione e possesso effettivo del dato.

Possibili strategie di contratto durante una migrazione:

- **contratto debole**: il client puo' osservare NOT_FOUND finche' il riallineamento non e' completato;
- **redirect esplicito**: il router consulta anche la posizione reale;
- **blocco temporaneo**: alcune operazioni sono sospese durante il rebalance;
- **protocollo a fasi**: routing nuovo visibile solo a migrazione conclusa.

Il laboratorio adotta la prima opzione proprio per rendere il problema evidente.

Esperimento Da Fare In Aula

- 1 avviare due shard e scrivere alcune chiavi;
- 2 osservare WHERE e GET;
- 3 aggiungere un terzo shard con ADD_SHARD;
- 4 osservare di nuovo WHERE e GET;
- 5 eseguire REBALANCE;
- 6 verificare che letture e routing tornino coerenti.

La sequenza serve a distinguere chiaramente:

- cambio di mappa;
- perdita di trasparenza;
- convergenza finale.

- chi decide quando la nuova topologia diventa osservabile dal client?
- e' accettabile che GET fallisca dopo un cambio di topologia?
- dobbiamo privilegiare disponibilita' o trasparenza semantica?
- come cambierebbe tutto questo con scritture concorrenti durante il rebalance?
- cosa succede se un nodo cade dopo IMPORT_KEY ma prima di DELETE_LOCAL?

Il rebalancing non e' un dettaglio di manutenzione. E' il primo punto del percorso in cui:

- la topologia diventa parte della semantica osservabile;
- il routing da solo non basta a spiegare il comportamento del sistema;
- l'interfaccia deve dichiarare cosa promette durante gli stati transitori.

Da qui in poi non basta piu' dire "so dove dovrebbe stare una chiave". Bisogna poter dire anche "so quando il sistema e' autorizzato a comportarsi come se fosse gia' li".